Aaron Ludwig A. Altar
CPE4B

Activity #4: Bills Counting
Date: Sept. 27, 2026

Code Snippets

```python
# Bill Color Detection
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
image = cv2.imread('Images/bills.png')
if image is None:
    raise FileNotFoundError("Image not found.")

image = cv2.resize(image, (1448, 1086))  # keep original resolution (resize if needed)

# Convert to RGB (for display) and HSV (for color detection)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
h_img, w_img = image.shape[:2]
```

[1]

```python
bill_colors = {
    'Orange': {'ranges': [((3,   150, 150), (15,  255, 255))],
               'box_color': (255, 140, 0),   'value': 20},
    'Yellow': {'ranges': [((17,  100, 100), (30,  255, 255))],
               'box_color': (230, 200, 0),   'value': 500},
    'Green':  {'ranges': [((38,  60,  50),  (75,  255, 255))],
               'box_color': (0, 150, 0),     'value': 200},
    'Blue':   {'ranges': [((95,  60,  50),  (118, 255, 255))],
               'box_color': (0, 0, 255),     'value': 1000},
    'Violet': {'ranges': [((119, 50,  50),  (160, 255, 255))],
               'box_color': (148, 0, 211),   'value': 100},
    'Red':    {'ranges': [((170, 110, 150), (180, 255, 255))],
               'box_color': (220, 0, 60),    'value': 50},
}

def build_mask(hsv_img, ranges):
    """Combine one or more HSV ranges (red-like hues can wrap around 0/180) into a single mask."""
    mask = np.zeros(hsv_img.shape[:2], dtype=np.uint8)
    for lower, upper in ranges:
        m = cv2.inRange(hsv_img, np.array(lower), np.array(upper))
        mask = cv2.bitwise_or(mask, m)
    return mask

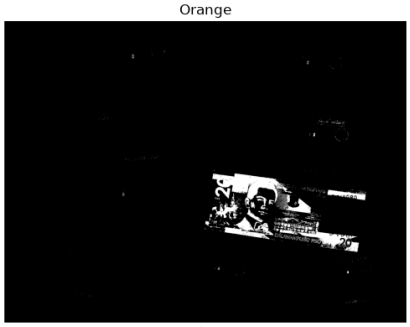
# Preview the raw per-color mask for every denomination
fig, axs = plt.subplots(2, 3, figsize=(15, 8))
for ax, (label, info) in zip(axs.flatten(), bill_colors.items()):
    mask = build_mask(hsv, info['ranges'])
    ax.imshow(mask, cmap='gray')
    ax.set_title(label)
    ax.axis("off")
plt.tight_layout()
plt.show()
```

[5]

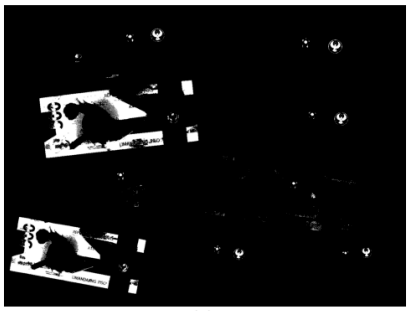
Aaron Ludwig A. Altar
CPE4B

Activity #4: Bills Counting
Date: Sept. 27, 2026

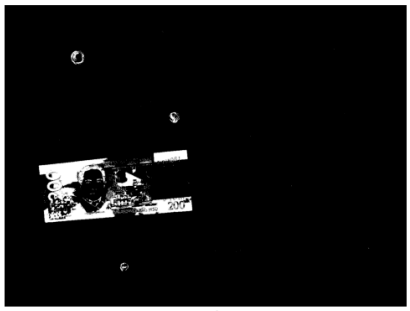
Orange



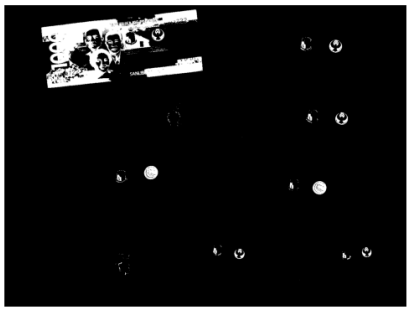
Yellow



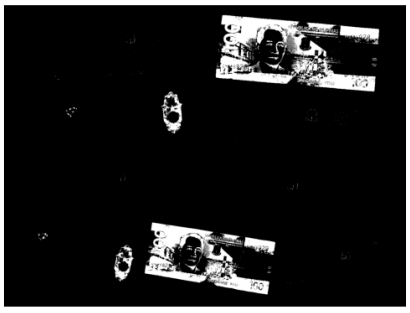
Green



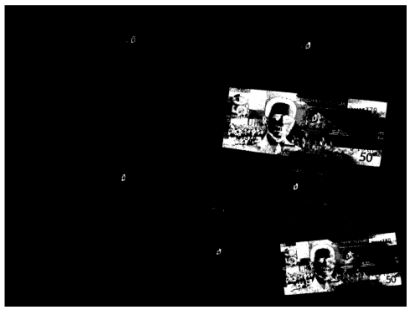
Blue



Violet



Red



Aaron Ludwig A. Altar
CPE4B

Activity #4: Bills Counting
Date: Sept. 27, 2026

```python
def merge_overlapping_boxes(boxes):
    """A single bill sometimes breaks into two touching/overlapping blobs
    (its photo/portrait can split the colored banner in half). Merge any
    boxes of the same color that overlap into one enclosing box so we
    don't count one bill twice."""
    boxes = [list(b) for b in boxes]
    merged = True
    while merged:
        merged = False
        for i in range(len(boxes)):
            for j in range(i + 1, len(boxes)):
                xi, yi, wi, hi = boxes[i]
                xj, yj, wj, hj = boxes[j]
                overlap_x = xi < xj + wj and xj < xi + wi
                overlap_y = yi < yj + hj and yj < yi + hi
                if overlap_x and overlap_y:
                    x = min(xi, xj)
                    y = min(yi, yj)
                    w = max(xi + wi, xj + wj) - x
                    h = max(yi + hi, yj + hj) - y
                    boxes[i] = [x, y, w, h]
                    del boxes[j]
                    merged = True
                    break
            if merged:
                break
    return [tuple(b) for b in boxes]

# Scale the morphology kernel and the minimum blob size to the image resolution
k_size = max(15, int(min(h_img, w_img) * 0.03))
if k_size % 2 == 0:
    k_size += 1
close_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k_size, k_size))
noise_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
min_area = int(h_img * w_img * 0.01)  # ignore blobs smaller than ~1% of the image

detections = []  # (label, value, (x, y, box_w, box_h))

for label, info in bill_colors.items():
    mask = build_mask(hsv, info['ranges'])
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, noise_kernel)              # remove speckle noise
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, close_kernel, iterations=2)  # bridge gaps
    mask = cv2.dilate(mask, close_kernel, iterations=1)                      # grow to full bill size

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    boxes = [cv2.boundingRect(c) for c in contours if cv2.contourArea(c) >= min_area]
    boxes = merge_overlapping_boxes(boxes)  # collapse a split bill back into one box

    for box in boxes:
        detections.append((label, info['value'], box))

print(f"Detected {len(detections)} bill(s).")
```

```
Detected 9 bill(s).
```

Aaron Ludwig A. Altar
CPE4B

Activity #4: Bills Counting
Date: Sept. 27, 2026

```python
# Draw a labeled box on every detected bill
output_img = image_rgb.copy()

for label, value, (x, y, box_w, box_h) in detections:
    color = bill_colors[label]['box_color']
    cv2.rectangle(output_img, (x, y), (x + box_w, y + box_h), color, 2)

    text = f"{label} - {value} Peso"
    (text_w, text_h), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
    text_y = max(y - 8, text_h + 4)
    cv2.rectangle(output_img, (x, text_y - text_h - 6), (x + text_w + 6, text_y + 4), (255, 255, 255), -1)
    cv2.putText(output_img, text, (x + 3, text_y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

plt.figure(figsize=(10, 8))
plt.imshow(output_img)
plt.title("Detected Bills")
plt.axis("off")
plt.show()

# Summary + total peso value
print("Detected bills:")
total_value = 0
for label, value, box in detections:
    print(f"  {label}: {value} Peso")
    total_value += value

print(f"\nTotal count: {len(detections)}")
print(f"Total value: P{total_value}")
```

Aaron Ludwig A. Altar **Activity #4: Bills Counting**

CPE4B **Date: Sept. 27, 2026**




```
Detected bills:
  Orange: 20 Peso
  Yellow: 500 Peso
  Yellow: 500 Peso
  Green: 200 Peso
  Blue: 1000 Peso
  Violet: 100 Peso
  Violet: 100 Peso
  Red: 50 Peso
  Red: 50 Peso

Total count: 9
Total value: P2520
```

Github Repository Link:

[Activity#4 - Bills_Counting Github Link]